Studio Nx

Backend(FastAPI/Python) 55 — retention.main()

2026-10-08

Backend(FastAPI/Python) 55 — `retention.main()` (+ 처리방침 버전 2026-10-08)

항목	내용
파일	`studionx/backend/app/workers/retention.py` / 처리방침: `privacy.html` · `home.html` · `join.js` · `schemas/auth.py` · `docs/Privacy_Policy_Draft.md`
작업	P1-12 문의 API — 보관 기간 정리를 하루 한 번 실행하는 명령 · 처리방침 §4
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	54 `purge_old_inquiries()` · (P1-12 마지막)

1. 하는 일

명령 한 줄로 **보관 기간 정리 두 가지**를 차례로 돌리고 결과를 한 줄 출력한다.

```
cd studionx/backend
..\.venv\Scripts\python -m app.workers.retention
# → withdrawn accounts: 0, inquiries: 0
```

차례	정리	함수
①	탈퇴 3일 지난 계정의 개인 정보 비우기	49 `purge_withdrawn_accounts` (P1-09)
②	답변 1년 · (답변 없으면) 받은 뒤 1년 지난 문의 지우기	54 `purge_old_inquiries`

정리 중 오류가 나면 예외를 그대로 올린다 → Sentry 로 보고되고, 명령은 실패로 끝난다(예약 작업이 실패를 알 수 있게).

2. 언제 쓰나

하루 한 번 운영체제의 예약 작업으로 돌린다 — Windows 작업 스케줄러 또는 리눅스 cron. 예약은 **배포할 때** 정한다 (템플릿 머리말). 지금은 손으로 한 번 실행해 동작을 확인했다 (실제 DB · Sentry 끔 → `withdrawn accounts: 0, inquiries: 0`, 종료 코드 0 — 지울 것이 없는 상태).

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import delete, func, or_, select
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.core.sentry import init_sentry
from app.db.session import SessionLocal
from app.models import Inquiry, User
```

```
from app.models.user import UserStatus
from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn
```

`init_sentry` · `SessionLocal` 줄을 이번에 추가했다.

함수

```
def main() -> None:
    init_sentry(get_settings())
    now = datetime.now(UTC)
    with SessionLocal() as db:
        accounts = purge_withdrawn_accounts(db, now)
        inquiries = purge_old_inquiries(db, now)
    print(f"withdrawn accounts: {accounts}, inquiries: {inquiries}")
```

파일 맨 아래의 `if __name__ == "__main__": main()` 은 템플릿 그대로 — `python -m app.workers.retention` 으로 실행하면 `main()` 이 불린다.

문법 풀이

import 문법	뜻
`from app.core.sentry import init_sentry`	서버(P1-01)와 같은 Sentry 설정 — 명령으로 따로 돌 때도 오류를 보고하려고
`from app.db.session import SessionLocal`	DB 세션 공장 — 서버 요청 밖에서 직접 세션을 만듦

식	뜻
`with SessionLocal() as db:`	세션을 열고, 블록이 끝나면(오류가 나도) 닫음
`datetime.now(UTC)`	시간대가 붙은 지금 시각 — 49 · 54 가 UTC 로 바꿔 비교
`f"…{accounts}…"`	글자 안에 값 끼우기
`python -m app.workers.retention`	파일을 **모듈**로 실행 — `app` 패키지를 찾을 수 있게 `studionx/backend` 에서

4. 흐름

1. Sentry 켬.
2. 지금 시각을 한 번 잼.
3. 세션을 열고 탈퇴 계정 정리 → 문의 정리 (각자 저장).
4. 결과 한 줄 출력. 오류면 그대로 올라감.

5. 왜 이렇게

선택	이유
기준 시각을 한 번만	두 정리가 같은 "지금" 을 기준으로 — 기록을 읽을 때 헷갈리지 않게
오류를 삼키지 않음	정리가 조용히 실패하면 처리방침의 약속(삭제)을 어기게 된다. Sentry 보고 + 실패 종료 코드로 알 수 있게
Sentry 를 먼저	그 뒤의 어떤 오류든 보고되게
한 줄 출력	예약 작업 기록에서 날마다 몇 개를 지웠는지 볼 수 있다

6. 처리방침 버전 올림 (사용자 결정 A, 2026-10-08)

54 의 결정("답변하지 않은 문의는 받은 뒤 1년")을 처리방침에 넣고, privacy.html 의 규칙("내용을 바꾸면 버전도 올림")대로 버전을 `2026-10-05` → **`2026-10-08`** 로 올렸다. 아직 공개 전(Draft)이라 다시 동의를 받을 사람은 없고, 앞으로 가입하는 사람은 새 버전으로 기록된다.

파일	바꾼 것
`privacy.html` §4	"Messages you send us — 1 year after we reply. **If we never reply, 1 year after you send them.**" · Version 줄
`home.html` 문의 요약	"One year after we reply — **or one year after you send it, if we never reply** — then we delete it." · Version 줄
`join.js` · `schemas/auth.py`	`PRIVACY_VERSION = '2026-10-08'` (가입 때 보내고 서버가 확인하는 값 — 둘이 같아야 가입됨)
`docs/Privacy_Policy_Draft.md`	상태 · §2.4 보관 표 · 버전 줄 + **버전 기록 표** (10-05 첫 확정판 / 10-08 문의 보관) + PDF
`tests/test_auth_service.py`	가입 시험 본문의 `"2026-10-05"` 를 상수 `PRIVACY_VERSION` 으로 (버전이 바뀌어도 시험이 따라감)

7. 시험 (`tests/test_retention.py` 에 2개 추가 · 모두 13개)

시험	확인한 것
`main()` (세션 · Sentry 를 시험용으로 바꿔 끼움)	Sentry 를 먼저 켬, 탈퇴 10일 계정 · 400일 문의를 정리, 출력 `withdrawn accounts: N, inquiries: M`
정리 중 오류	예외가 그대로 올라옴
(손으로) 실제 명령	`withdrawn accounts: 0, inquiries: 0`, 종료 코드 0

전체 시험 405개 통과 (처리방침 버전을 올린 뒤에도), `ruff` 검사 통과.